HPC Project 3: Using MPI for Computing Eigenvalues of Large Random Matrices

Joe Ruse

April 21, 2026

Contents

1 Introduction **2**

2 Methods **3**
2.1 Random Matrix Generation . . . 3
2.2 Computing the Maximal Eigenvalue . . . 3
2.3 Manager-Worker Pattern . . . 4
2.3.1 Manager . . . 4
2.3.2 Worker . . . 4

3 Results **7**
3.1 Performance . . . 7
3.1.1 Ensemble Size Scaling . . . 7
3.1.2 Matrix Size Scaling . . . 8
3.1.3 Parallel Workers . . . 8
3.2 Eigenvalue Distributions . . . 9

4 Conclusion **12**

1 Introduction

The large-scale collection of data in the Digital Age has placed linear algebra at the core of modern computing. Matrix operations are now a foundational component in applications ranging from video games and computer graphics to machine learning and financial modelling. As these applications grow in scale, so too does the demand for efficiently handling large matrix operations.

In this project, we investigate the computation of maximal eigenvalues of large random matrices, focusing particularly on the computational cost of these operations and the constraints of performing them at scale. For an ensemble of M matrices of size $N \times N$, the memory requirements scale as $\mathcal{O}(MN^2)$, while the computation time scales as $\mathcal{O}(MN^3)$. These scaling laws impose a practical limitation: the large matrix sizes and ensemble counts that are required to resolve subtle statistical features quickly exceed the capabilities of single-node, shared-memory systems.

These limitations are especially significant in the context of random matrix theory. The matrices considered here are drawn from the Gaussian Orthogonal Ensemble, consisting of real, symmetric matrices with entries sampled from Gaussian distributions with mean $\mu = 0.0$, diagonal variance of $\sigma^2_{i=j} = 4.0$, and off-diagonal variance $\sigma^2_{i\neq j} = 1.0$. While the bulk eigenvalue distribution of such matrices is well understood, the statistical behaviour of the largest eigenvalue is more delicate and requires both large matrix dimensions and large ensembles to resolve accurately.

A key result is the emergence of the Tracy–Widom distribution, introduced by Craig Tracy and Harold Widom, which describes the limiting distribution of the largest eigenvalue after appropriate normalization. The distinction between this distribution and a Gaussian is subtle, primarily existing in the tails and only evident when statistical noise is sufficiently reduced. The need to generate a large number of rare events requires those exact same large-scale computationally demanding operations discussed above.

To address the high demands of these computations, we employ distributed-memory parallelism with MPI using a Manager-Worker pattern, allowing for the matrices and their associated computations to be assigned to individual workers partitioned across multiple processes. This approach enables us to scale beyond the limitations of a single workstation, making it feasible to generate large enough ensembles to probe the Tracy-Widom regime.

The maximal eigenvalues are computed primarily using the power iteration method, which offers favourable computational scaling for large matrices. However, because convergence is not guaranteed within a fixed number of iterations, a fallback based on the Schur decomposition is used to ensure robustness without introducing bias into the ensemble.

Finally, to determine whether the empirical eigenvalue distribution deviates from normality and approaches the Tracy–Widom form, we apply statistical hypothesis testing. The Shapiro–Wilk test is used to assess normality, while the Kolmogorov–Smirnov test is used to compare the empirical distribution against a reference. Together, these tools allow us to quantitatively evaluate whether the computational effort successfully captures the expected limiting behaviour.

2 Methods

2.1 Random Matrix Generation

For each sample, a real symmetric matrix $A \in \mathbb{R}^{N\times N}$ is generated from the Gaussian Orthogonal Ensemble (GOE). The entries are sampled independently up to the symmetry constraint that $A_{ij} = A_{ji}$. Main diagonal entries are drawn from the Gaussian distribution with mean $\mu = 0.0$ and standard deviation $\sigma_{i=j} = 2.0$, while off-diagonal entries are sampled with a standard deviation of $\sigma_{i\neq j} = 1.0$. In a more compact form,

$$A_{ij} \sim \begin{cases} \mathcal{N}(\mu = 0.0, \sigma^2 = 4.0) & i = j \\ \mathcal{N}(\mu = 0.0, \sigma^2 = 1.0) & i \neq j. \end{cases} \tag{1}$$

Each matrix is real and symmetric, which guarantees the eigenvalues to be real numbers. This makes the GOE particularly suitable for studying the statistics of maximal eigenvalues. The matrix generation is performed by each MPI worker using the VSL Gaussian random number generator from the Intel MKL library, with a unique seed provided by the MPI manager when work is assigned.

2.2 Computing the Maximal Eigenvalue

The quantity of interest is the largest magnitude eigenvalue $|\lambda|_{\max}$ of each random matrix. For each sample, we first attempt to approximate $|\lambda|_{\max}$ using the power iteration method. Given an initial normalized vector v_0, the method computes the sequence

$$v_{k+1} = \frac{Av_k}{\|Av_k\|}, \tag{2}$$

where v_{k+1} eventually converges to the eigenvector $v_{\max}$ that corresponds to the largest magnitude eigenvalue $|\lambda_{\max}|$, provided that the initial vector has a nonzero component in that direction and that the dominant eigenvalue is sufficiently separated from the rest of the spectrum.

In each iteration, the associated eigenvalue is approximated using the Rayleigh quotient,

$$\lambda_k = v_k^T A v_k. \tag{3}$$

Convergence is monitored by comparing successive eigenvalue estimates, considering convergence to be achieved when the relative difference $\frac{|\lambda_{k+1}-\lambda_k|}{|\lambda_k|} < \epsilon = 10^{-5}$. Since the power method naturally converges to the eigenvalue of largest magnitude rather than necessarily the largest algebraic eigenvalue, the implementation applies a spectral shift when needed. If the dominant eigenvalue in magnitude is negative, the iteration is repeated for the shifted matrix $A + \mu I$, with $\mu = -\lambda$, and the shift is then removed from the final estimate. This allows the method to target the largest algebraic eigenvalue rather than the most negative extreme of the spectrum.

Power iteration is used here because each step is dominated by matrix-vector products, giving a per-iteration cost of $\mathcal{O}(N^2)$, which is significantly cheaper than a full eigenvalue decomposition. However, convergence is not guaranteed within a finite number of iterations.

In cases where the power iteration fails to converge within $N/2$ iterations, a more robust method from LAPACK based on the Schur decomposition is used. Schur decomposition states that any real matrix A can be factorized as

$$A = QTQ^T$$

where Q is an orthogonal matrix and T is upper quasi-triangular. For real symmetric matrices, this decomposition simplifies further, and the matrix T becomes diagonal, with its entries corresponding directly to the eigenvalues of A.

Unlike the power iteration, the Schur-based approach is guaranteed to succeed for all matrices, but at a significantly higher computational cost, scaling as $\mathcal{O}(N^3)$. For this reason, it is used only as a fallback method to ensure that all samples contribute to the ensemble without introducing bias. Thus, using power iteration with the Schur fallback, we can expect the wall times to scale with matrix size between $\mathcal{O}(N^2)$ in the best case and $\mathcal{O}(N^3)$ in the worst case.

2.3 Manager-Worker Pattern

The computation is parallelized using MPI in a manager–worker configuration. One process (rank 0) acts as the manager, while all remaining processes act as workers. The problem is naturally suited to this model, as each realization of the random matrix ensemble is independent and can be computed without communication between workers.

At initialization, the manager reads the input parameters and broadcasts the matrix size N to all workers. Each worker then allocates memory for a single $N \times N$ matrix. The computation proceeds as a sequence of independent tasks, where each task consists of generating a random matrix and computing its maximal eigenvalue.

The manager is responsible for distributing work, collecting results, and determining when the computation is complete. Workers repeatedly await work orders, perform the assigned computation, and return the result. Communication between processes is handled using MPI message passing, with tags used to distinguish between work assignments and termination signals.

To maintain efficiency, the manager tracks the availability of each worker and dynamically assigns new tasks as workers become free. Once the required number of eigenvalues has been collected, the manager stops issuing new work and instead sends termination signals. The program terminates only after all workers have exited cleanly.

This design ensures good load balancing and minimizes idle time, while also avoiding the need to store large ensembles of matrices in memory simultaneously.

2.3.1 Manager

The manager process is outlined in Algorithm 1. It handles the distribution of work, collection of results, and terminating workers once the desired number of eigenvalues has been obtained. It maintains a record of which workers are busy with tasks and dynamically assigns new tasks as workers complete their computations. Each work order consists of a random seed used by the worker to generate a matrix, and an operation tag which tells the worker to either perform a computation or to exit. Results are received asynchronously, allowing the manager to immediately reassign work to idle workers. Once the desired number of results have been collected, the manager sends termination signals to all workers and exits after all workers have been dismissed.

2.3.2 Worker

Each worker process (shown in Algorithm 2) begins by receiving the broadcast matrix size N from the manager and allocating memory for a single $N \times N$ matrix. The worker then enters a loop in which it waits for a message from the manager. Each message contains a random seed and an MPI tag.

If the received tag is an exit tag, the worker leaves the loop and terminates. Otherwise, the worker initializes its random number stream using the provided seed, generates a GOE matrix, and computes its maximal eigenvalue. The primary method is power iteration, with a dense LAPACK eigenvalue routine used as a fallback if convergence is not achieved within the allowed number of iterations. The

Algorithm 1 Manager

Require: manager = Proc 0, workers = Proc $1, \ldots, n_{\text{procs}} - 1$
Require: matrix size N, target ensemble size n_{eig}

```
broadcast N to all workers
initialize counters: received = 0, failed = 0, exitTags = 0
mark all workers as available
while exitTags < number of workers do
    for each available worker do
        if received < required results (including in-flight work) then
            generate random seed
            send seed with DEFAULT tag to worker
        else
            send EXIT tag to worker
            increment exitTags
        end if
        mark worker as busy
    end for
    if exitTags < number of workers then
        receive (λ_max, status) from any worker
        if status indicates success then
            store λ_max
            increment received
        else
            increment failed
        end if
        mark sending worker as available
    end if
end while
```

worker then sends the resulting eigenvalue back to the manager together with a tag indicating whether the primary iterative solve succeeded. The worker repeats this process until explicitly dismissed by the manager.

Algorithm 2 Worker

Require: worker = Proc $1, \ldots, n_{\text{procs}} - 1$

```
receive broadcast matrix size N
allocate matrix A ∈ ℝ^{N×N}
while True do
    receive seed and tag from manager
    if tag = EXIT then
        exit loop
    end if
    initialize random number generator with seed
    generate GOE matrix A
    compute λ_max using power iteration
    if power iteration fails then
        compute λ_max using Schur decomposition
        set status = fallback
    else
        set status = success
    end if
    send (λ_max, status) to manager
end while
deallocate A
```

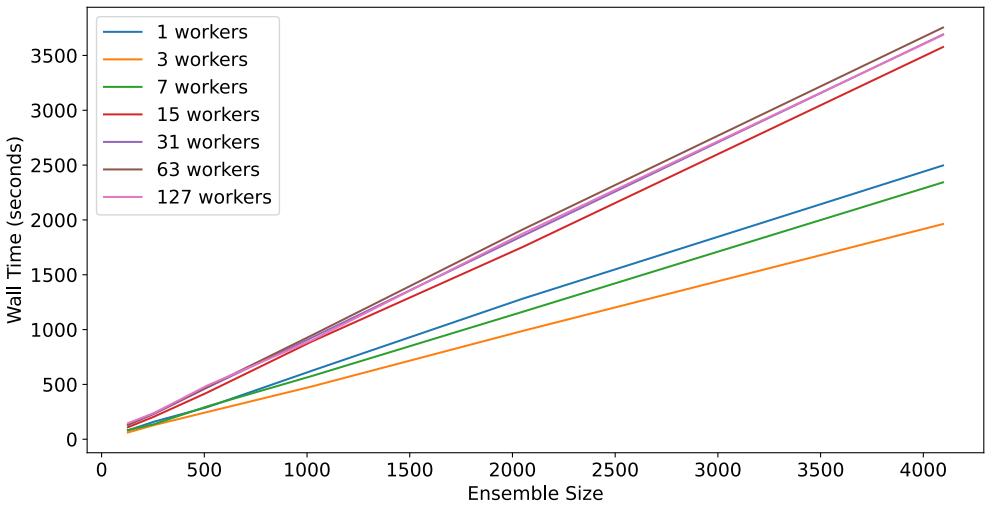

Figure 1: Wall times vs ensemble size across different numbers of workers with matrix size $N = 1500$. Wall times are seen to scale approximately linearly, with slopes dependent on the number of workers. Note that the 31 worker line shown in purple is obscured by the 127 worker line in pink.

3 Results

3.1 Performance

To explore the performance scaling of the computation, wall times were recorded for various matrix sizes, ensemble sizes, and numbers of workers.

3.1.1 Ensemble Size Scaling

The scaling of wall time with ensemble size is seen in Figure 1. While the recorded wall times approximately match the expected linear scaling, some deviation is seen at lower ensemble sizes where overhead time is not dominated by computation time.

An interesting result is seen in this scaling when comparing the different numbers of workers. While it would be expected that more workers would result in lower times, we can see that the fastest configuration was with only 3 workers. At an ensemble size of $M = 2048$, the 3-worker time $w_3 = 986.94$s is approximately 77.1% of the 1-worker time $w_1 = 1280.58$s, corresponding to a little more than a 1.3× speedup. In contrast, the 7-worker time $w_7 = 1161.19$s is 117.7% longer than w_3.

In addition, we also see that the 1-3-7 group is clearly separated from the rest of the lines. This distinct grouping comes as a result of the hardware used to perform the operations which had only 12 CPU cores. By exceeding the number of available cores, each worker must then share compute time with others, leading to the nearly doubled wall times for greater than 12 workers. This result highlights the need to distribute such work over several nodes, as single node systems are quickly limited in scope when it comes to parallelism at large scales.

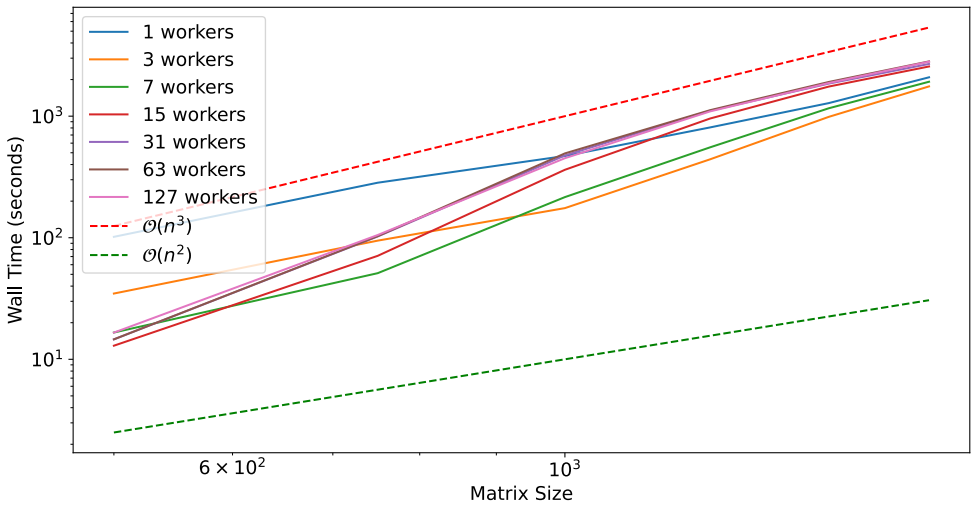

Figure 2: Wall time vs matrix size N for each number of workers for ensemble size $M = 2048$, with the best case scaling of $\mathcal{O}(N^2)$ shown as a green dashed line and worst case (Schur fallback) $\mathcal{O}(N^3)$ scaling as the red dashed line.

3.1.2 Matrix Size Scaling

In figure 2, the scaling of wall times with matrix size are shown for each number of workers at a fixed ensemble size $M = 2048$. In this figure we see that the wall times stay between the best and worst case scaling of $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$ respectively.

At high matrix sizes, the same unordered hierarchy shown in figure 1 is seen. However, at lower matrix sizes, we see an ordering of wall times that fits more with the expectation of adding workers up to 15 workers. For > 15 workers, we see that the wall times grow larger again, following the explanation given above.

The reason that the worker numbers that exceed the hardware capabilities are still faster than 1 and 3 workers at these low matrix sizes is likely that the computation time for one matrix is small enough that excess workers are not idle for long between solutions. As the matrix size increases, the computation of a single eigenvalue takes longer, making the excess workers wait longer and thus the wall times for excess numbers of workers to be longer.

3.1.3 Parallel Workers

In figure 3, the wall times for different matrix sizes are plotted against increasing numbers of workers. These plots highlight the behaviour seen in cases where the number of workers is greater than the number of CPU cores.

For small matrices ($N = 500$ and 700), we see that increasing the number of workers continues to reduce the wall time. At $N = 750$, we begin to see a non-monotonic relationship, where the initial gains of moving from 1 to 3 workers are undone by increasing the number of workers, but still run faster than a single worker. For matrix size $N = 1000$ and above, exceeding the number of CPU cores results in wall times greater than if a single worker had been used.

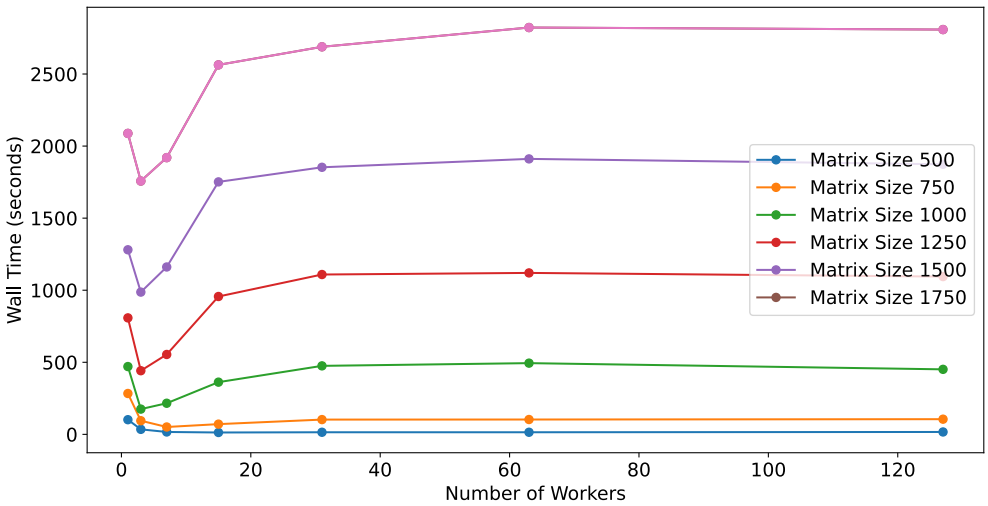

Figure 3: Wall times vs number of workers at a fixed ensemble size $M = 2048$ for different matrix sizes.

3.2 Eigenvalue Distributions

The Tracy-Widom distribution was approximated using several different matrix and ensemble sizes. Here, we explore how these parameters affect the approximation. In all plots, the eigenvalues are normalized to their mean, and the frequencies normalized to be a probability density.

Figure 4 shows the Tracy-Widom approximations from various matrix sizes. In all cases, the ensemble size is $M = 56488$, the combined total of all computations for each matrix size.

The role that matrix size plays in the approximation is clearly displayed here. Smaller matrices result in tighter eigenvalue distributions, while larger ones have wider distributions. This result shows that large matrices are required to capture a sufficient number of samples in the tails in order to properly approximate the Tracy-Widom distribution. Based on this result, the matrix size $N = 1750$ will be used for the subsequent investigation.

In figure 5, the approximated results are shown at a fixed matrix size $N = 1750$ for increasing ensemble size. At low ensemble sizes, the histogram of eigenvalues is noisy and does not have enough data to fill in the rest of the distribution. As ensemble size grows, we see that the noise decreases, but the location of the distribution shifts. This result highlights the noise-reduction role that ensemble size plays in the approximation. By using large ensembles we can smooth out the noise and variance, improving the agreement with the Tracy-Widom.

At $M = 7168$ and above, we can see that the histogram has the approximate shape of the Tracy-Widom distribution but there is some variance in the location of the peak. By combining all results, we should then expect to see a close approximation to Tracy-Widom.

Figure 6 shows the combined result of all $N = 1750$ eigenvalues, yielding an ensemble of $M = 51200$ eigenvalues. With this large sample size, we see a close approximation to the Tracy-Widom distribution.

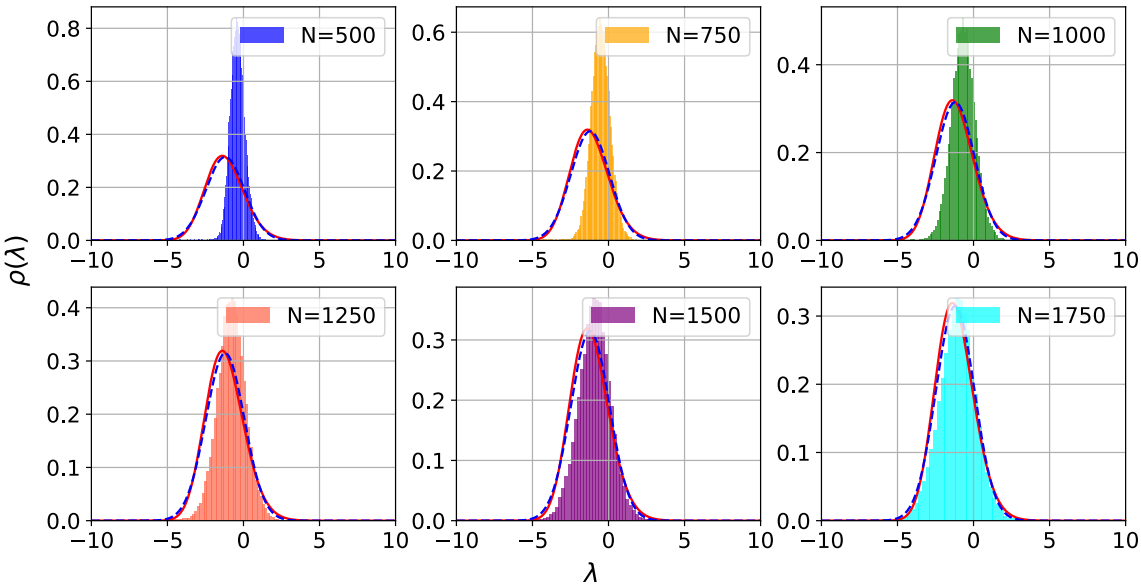

Figure 4: Maximal eigenvalue distribution for different matrix sizes with ensemble size $M = 56488$. For $N = 1750$, the ensemble size is $M = 51200$ due to an interrupted test. The normal and Tracy-Widom distributions are shown in blue and red respectively.

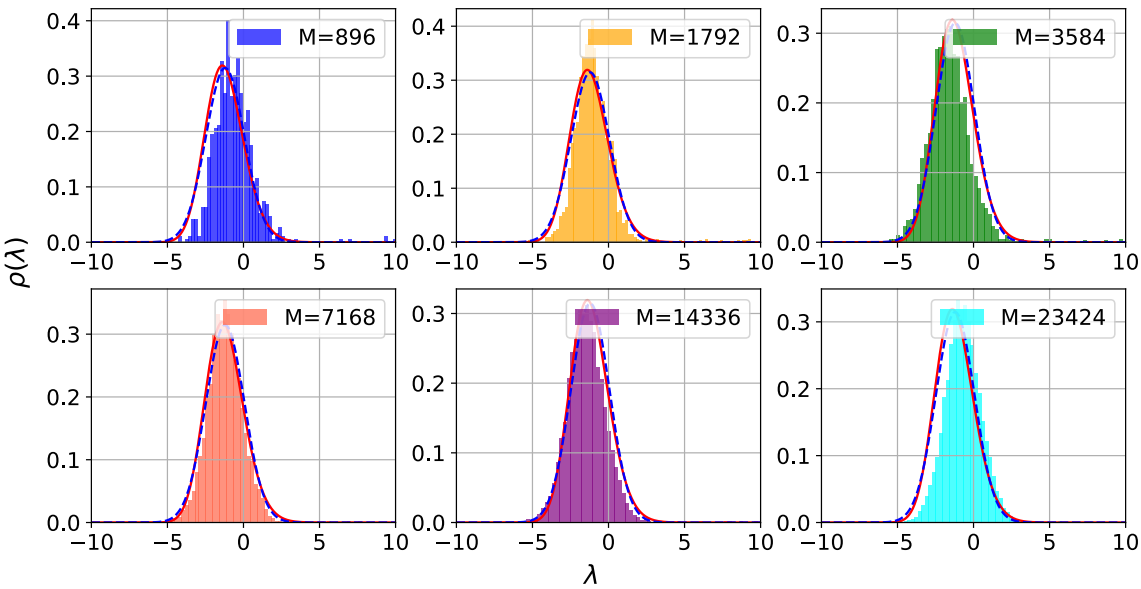

Figure 5: Distribution of maximal eigenvalues with matrix size $N = 1750$ for various ensemble sizes. Normal distribution is shown in blue and Tracy-Widom in red.

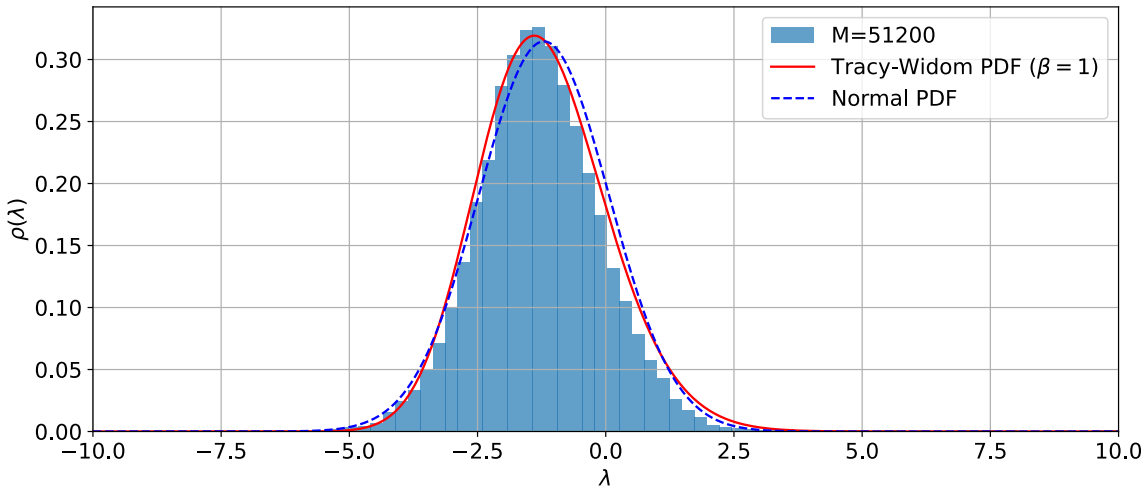

Figure 6: Combined result of all $N = 1750$ distributions for a total ensemble size of $M = 51200$.

M	S-W	S-W p-value	K-S (TW)	K-S (TW) p-value	Skewness	Kurtosis
896	0.35	4.25×10^{-18}	0.214	1.24×10^{-36}	4.80	23.7
1792	0.243	1.37×10^{-26}	0.110	1.79×10^{-19}	5.56	34.7
3584	0.305	1.39×10^{-34}	0.085	3.75×10^{-23}	4.74	23.2
7168	0.291	7.38×10^{-46}	0.073	1.25×10^{-33}	5.14	27.9
14336	0.288	4.22×10^{-59}	0.079	6.22×10^{-80}	4.87	24.7
23424	0.271	3.59×10^{-70}	0.220	0.00×10^{00}	4.98	26.2
51200	0.283	9.43×10^{-89}	0.079	4.59×10^{-276}	4.99	26.2

Table 1: Results of Shapiro-Wilk and Kolmogorov-Smirnov tests for increasing ensemble size.

Visually, we can see that the left side of the distribution closely follows the Tracy-Widom shown in red. On the right, our approximation does not quite fill in up to the theoretical line, but is clearly closer to the Tracy-Widom than to the normal distribution. We can evaluate the approximation's closeness to normality and the target distribution quantitatively using the Shapiro-Wilk test and Kolmogorov-Smirnov test respectively.

The results of the Shapiro-Wilk (SW) and Kolmogorov-Smirnov (KS) tests are shown in table 1. The Shapiro-Wilk test evaluates the null hypothesis that an ensemble is normally distributed. Smaller p-values for the SW test suggest against the null hypothesis. The Kolmogorov-Smirnov test evaluates an ensemble's agreement with a reference distribution by determining the separation between them. Here, a smaller K-S value indicates closer agreement between the sample and reference.

From the table, we can see that increasing ensemble size M greatly reduces the S-W p-value, indicating that the distribution moves away from normality as the ensemble grows. Similarly, we see that the K-S test yields smaller values as the ensemble grows, corresponding to the GOE case of the Tracy-Widom distribution ($\beta = 1.0$).

Based on these tests, we can see that the $M = 14336$ and $M = 51200$ ensembles have in the same K-S result. Additionally, at $M = 14336$ the S-W p-value is sufficiently small to suggest a non-normal distribution. Thus, for matrix size $N = 1750$, it appears that somewhere around 15000 samples is sufficient to approximate the Tracy-Widom distribution.

4 Conclusion

This project investigated the computation of eigenvalues of random matrices and their convergence to the Tracy-Widom distribution. This task was chosen because of its high cost of both memory and computation time, motivating the use of distributed-memory implementations.

The maximal eigenvalues of GOE matrices were computed using a robust method which first attempts to solve them using power iteration, and falls back on a guaranteed method based on Schur decomposition in the event that the power iteration fails to converge. The program as written in Fortran, and LAPACK functions were used for all vectorized operations.

The scaling of wall times with matrix size N were shown to fall within the expected range between $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$, and to grow linearly with ensemble size.

When comparing the scaling of wall time vs ensemble size for different numbers of workers, it was seen that the times and scaling were heavily dependent on the number of cores available in the CPU. While this emphasized the need for compute clusters, it also brought to light an interesting discovery about the workstation used. These experiments were performed on an Intel i7-14700k CPU. This processor is advertised as a 20-core CPU and reports as such using `lscpu` in the terminal. However, this project revealed that only 12 of these cores are what Intel calls 'performance' cores, which behave and operate as one would expect from a normal CPU core. The remaining 8 cores are called 'efficiency' cores, which are smaller and more power efficient, but are only used for small background tasks and day-to-day OS operations.

When evaluating the distribution of eigenvalues, it was shown that the matrix size plays an important role in filling the tails of the distribution. Larger matrices appear to yield a wider range of eigenvalues in the same amount of time as smaller matrices. The noise-reduction role of ensemble size was also shown, where small ensembles yielded noisy histograms, but larger ones produce smooth distributions. From our statistical tests, it was determined that a matrix size of $N = 1750$ can adequately approximate the Tracy-Widom using around 15000 samples, as the jump to over 51000 samples did not improve the K-S test results.

While the task performed in this project was relatively simple, it mirrors the same type of work done at scale in real-world applications. With CPU speeds remaining almost constant for nearly two decades, parallelism has become the main vector for scaling HPC infrastructure. This project has shown that even simple projects can quickly out scale the capabilities of single-node machines, highlighting the importance of distributed work loads when it comes to high performance computing.

References